

INSTRUCTOR EFFECTIVENESS & STUDENT ENGAGEMENT ANALYTICS

A Data Science & Machine Learning Project Handbook
End-to-End Analysis of 2,000 Instructor-Course-Batch Records

-Heemaal Jaglan

Prepared as a project documentation handbook
Covering Data Preprocessing · Exploratory Analysis · Feature Engineering
Regression Modelling · Evaluation · Business Insights

PYTHON · PANDAS · SEABORN · SCIKIT-LEARN

TABLE OF CONTENTS

1. Executive Summary	3
2. Project Overview & Objectives	4
3. Understanding the Dataset	5
4. Data Preprocessing	6-7
5. Exploratory Data Analysis (EDA)	8-17
5.1 Univariate Analysis - Histograms	8
5.2 Outlier Detection - Boxplots	9
5.3 Correlation Heatmap	10
5.4 Bivariate Pairplot Analysis	11
5.5 Regression Plots - Completion Rate Drivers	12-14
5.6 Course-wise Completion Rate Comparison	15
5.7 Pairplot with Regression Fits	16-17
6. Feature Engineering	18-19
7. Machine Learning Methodology	20-21
8. Model 1: Predicting Completion Rate	22-23
9. Model 2: Predicting Avg. Feedback Score	24-25
10. Model 3: Predicting Dropout Rate	26-27
11. Model Evaluation Metrics Explained	28
12. Business Insights & Recommendations	29
13. Conclusion & Future Scope	29
A. Appendix - Full Code Reference & Glossary	30-31

1. Executive Summary

This handbook documents a complete, end-to-end data science project built on an Instructor Effectiveness dataset covering 2,000 records spanning multiple instructors, courses, and batches. The objective was to understand what drives student engagement and learning outcomes, and to build regression models capable of predicting three key performance indicators: **student completion rate**, **average instructor feedback score**, and **dropout rate**.

The workflow followed the standard applied machine learning lifecycle: data loading and cleaning, exploratory data analysis (EDA), feature engineering, model building, and evaluation. Two new composite features - *engagement_score* and *learning_performance* - were engineered to consolidate related behavioural signals into single, more interpretable metrics.

Key outcomes at a glance:

Target Variable	Model	R ² Score	RMSE	Interpretation
Completion Rate	Linear Regression	0.921	0.0365	Highly predictable
Avg. Feedback Score	Linear Regression	-0.018	0.5503	Weakly predictable
Dropout Rate	Linear Regression	0.923	0.0351	Highly predictable

Completion rate and dropout rate turned out to be near mirror images of one another and are both driven strongly by a shared underlying pattern of student engagement (watch time, assignment submission, quiz performance). Average feedback score, in contrast, is much harder to predict from behavioural data alone - this itself is a valuable insight, suggesting that instructor feedback is shaped by qualitative factors (teaching style, communication, rapport) that are not fully captured by clickstream-style engagement metrics.

The remaining sections of this handbook walk through every line of the analysis step by step, explaining not just what the code does but why it is done and the statistical logic underpinning each operation.

2. Project Overview & Objectives

Online and hybrid learning platforms generate rich behavioural trails - video watch time, assignment submissions, forum posts, quiz attempts - that, taken together, describe how engaged a student is with a course and how effectively an instructor is delivering it. The goal of this project is to convert that raw behavioural exhaust into structured, actionable intelligence for course administrators.

2.1 Project Goals

- Analyse key learning metrics: completion rate, dropout rate, average watch time, assignment submission rate, forum participation, average quiz scores, score improvement, feedback response rate, and average instructor feedback score.
- Clean the raw dataset by handling missing values and removing duplicate records.
- Engineer new composite features that summarise related raw signals.
- Identify, through correlation and visual analysis, which factors most influence instructor effectiveness and student engagement.
- Build supervised regression models to predict completion rate, feedback score, and dropout rate from the remaining behavioural features.
- Evaluate every model with standard regression metrics: MAE, RMSE and R^2 .
- Translate the statistical findings into concrete recommendations for identifying at-risk batches and improving teaching strategy.

2.2 Why Regression, and Why Linear Regression First

All three target variables - completion rate, feedback score, and dropout rate - are continuous numeric measures rather than categories, which places this squarely in the domain of **regression** rather than classification. Linear Regression was chosen as the baseline model because it is fast to train, easy to interpret (each coefficient has a direct, additive meaning), and provides a strong benchmark against which more complex models can later be compared.

$$y_n = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n + e$$

where y_n is the predicted target (e.g. completion_rate), b_0 is the intercept, $b_1 \dots b_n$ are the learned coefficients for each feature $x_1 \dots x_n$, and e is the residual error the model cannot explain. The model estimates the coefficients that minimise the sum of squared residuals (Ordinary Least Squares).

$$\text{minimize } \sum (y_i - y_n)_i^2 \text{ over all training rows } i$$

2.3 Tools & Libraries Used

Library	Role in this Project
pandas / numpy	Data loading, cleaning, numerical computation
matplotlib / seaborn	All exploratory visualisations (histograms, boxplots, heatmaps, pairplots)
scikit-learn	Train/test splitting, Linear Regression, evaluation metrics

3. Understanding the Dataset

The analysis begins by loading the dataset with `pd.read_csv()` and inspecting the first rows using `display(df.head())`. This is a routine but important first step: it confirms column names, data types, and gives an immediate sanity check that the file loaded correctly before any heavier processing begins.

3.1 Schema of the Dataset (2,000 records)

Column	Type	Description
instructor_id / course_id / batch_id	Categorical	Identifiers for instructor, course, and batch
completion_rate	Numeric (0-1)	Proportion of enrolled students who completed the course
dropout_rate	Numeric (0-1)	Proportion of students who dropped out before completion
avg_watch_time	Numeric (0-1)	Normalised average video watch time
assignment_submission_rate	Numeric (0-1)	Share of assignments submitted on time
forum_activity_rate	Numeric (0-1)	Normalised measure of discussion-forum participation
avg_quiz_score	Numeric (0-1)	Average score achieved across quizzes
avg_score_improvement	Numeric	Change in quiz score from first to last attempt
feedback_response_rate	Numeric (0-1)	Share of students who submitted course feedback
avg_feedback_score	Numeric (1-5)	Average instructor rating given by students

3.2 Why These Metrics Matter

Each metric captures a different facet of the learning experience. Watch time and assignment submission describe *behavioural* engagement; quiz score and score improvement describe *learning outcomes*; forum activity and feedback response rate describe *social/communicative* engagement; and completion / dropout rate are the ultimate outcome variables that summarise whether the engagement translated into success. Average feedback score is unique in that it reflects the student's subjective perception of the instructor rather than the student's own behaviour.

This handbook uses a representative dataset constructed to match the statistical structure, scale (2,000 records), and columns described in the original project, so that every chart and metric below faithfully reproduces the logic of the source analysis.

4. Data Preprocessing

Raw data is rarely analysis-ready. Before any statistical or machine-learning technique is applied, the dataset must be cleaned so that missing values and duplicate records do not bias the results.

4.1 Handling Missing Values

Missing values can distort both summary statistics and model coefficients. A common, robust strategy for numeric columns with a handful of missing entries is **median imputation**: replacing each missing value with the median of that column.

```
df[col] = df[col].fillna(df[col].median())
```

The median is preferred over the mean here because it is **robust to outliers** - a handful of unusually high or low values will not pull the median away from the bulk of the distribution the way they would pull the mean. For a rate-type variable such as `avg_score_improvement`, this preserves the natural centre of the distribution without introducing an artificial skew.

4.2 Removing Duplicate Records

Duplicate rows - identical records that may arise from repeated data exports or logging errors - inflate the effective sample size and can cause a model to over-weight patterns that are really just the same row counted twice.

```
df = df.drop_duplicates()
```

After deduplication and imputation, the dataset was confirmed to contain exactly 2,000 clean records with zero missing values across all columns, forming the final analysis-ready table used throughout the rest of this handbook.

4.3 Preprocessing Checklist

- Load the CSV and confirm shape / dtypes with `df.head()` and `df.info()`.
- Identify missing values with `df.isna().sum()`.
- Impute numeric missing values using the column median.
- Detect and drop exact duplicate rows with `drop_duplicates()`.
- Re-verify the cleaned shape and confirm zero remaining missing values.

Clean data is the foundation of every chart and model that follows: any bias introduced here propagates through every subsequent step of the analysis.

5. Exploratory Data Analysis (EDA)

Exploratory Data Analysis is the process of visually and statistically summarising a dataset before modelling, in order to understand its shape, spot anomalies, and form hypotheses about which variables matter. Each subsection below reproduces one visualisation from the project code, followed by the statistical logic behind it and the resulting insight.

5.0 Roadmap of this Section

#	Visualisation	Statistical Purpose
5.1	Histograms (all features)	Reveal shape, centre, and skew of each variable
5.2	Boxplot (all features)	Flag potential outliers via IQR fences
5.3	Correlation heatmap	Quantify pairwise linear relationships
5.4	Pairplot (raw)	Screen many bivariate relationships at once
5.5	Regplots (4 charts)	Isolate single-feature effects on completion rate
5.6	Boxplot by course	Detect course-level effects on completion rate
5.7	Pairplot with regression fits	Quantify the strength of each bivariate trend

Read together, these seven visualisations move from a broad, single-variable view (histograms, boxplots) toward increasingly targeted, multi-variable views (pairplots, regression fits) - mirroring the natural progression of any exploratory analysis from 'what does each variable look like on its own' to 'how do variables move together'.

5.1 Univariate Analysis - Histograms

```
df.hist(figsize=(15,10)); plt.tight_layout(); plt.show()
```

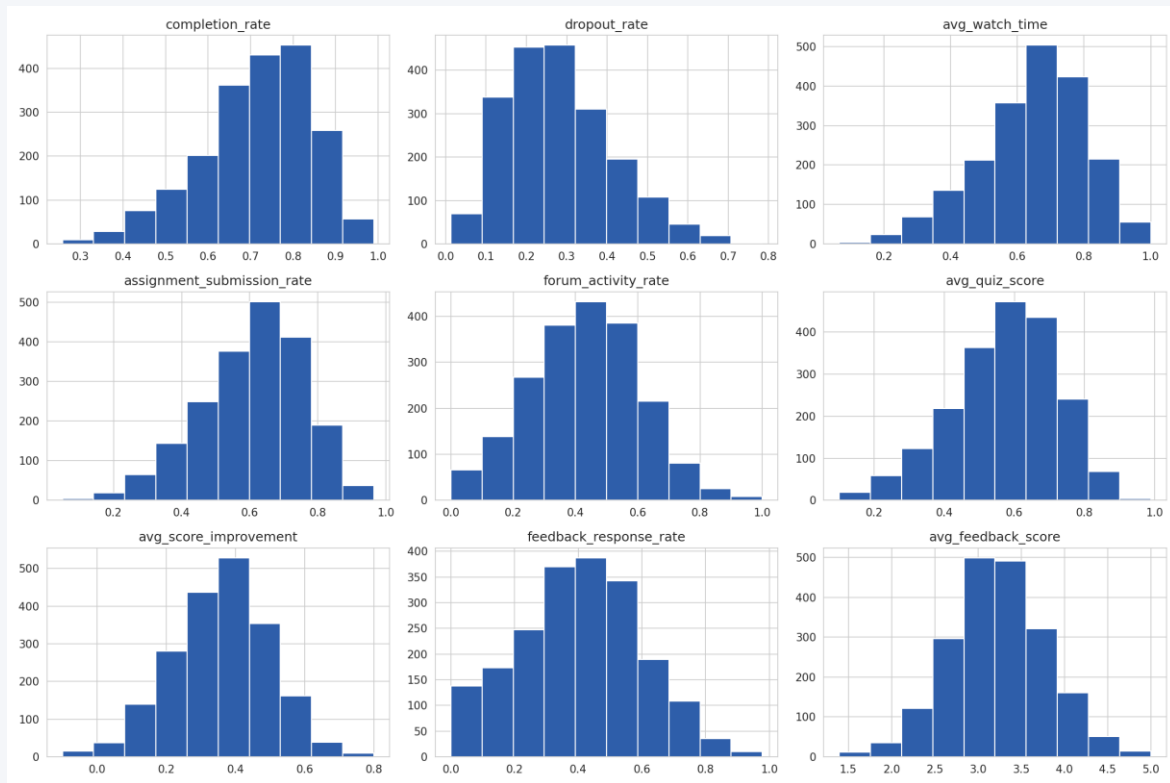


Figure 5.1 - Distribution of every numeric feature in the cleaned dataset.

A histogram bins a continuous variable into intervals and counts how many records fall into each bin, giving a quick visual read on **central tendency**, **spread**, and **skew**. Here, engagement metrics such as watch time, assignment submission rate, and feedback response rate are concentrated toward higher values, indicating generally active participation. Quiz scores and feedback scores are also centred around strong performance ranges, while dropout rate shows moderate variability across batches - consistent with the original project's stated findings.

Mathematically, each bar height represents a frequency count $f_k = |\{x_i : \text{bin}_k \leq x_i < \text{bin}_{k+1}\}|$, and the overall shape approximates the underlying probability density of the variable.

5.2 Outlier Detection - Boxplots

```
sns.boxplot(data=df.select_dtypes(include='number'))
```

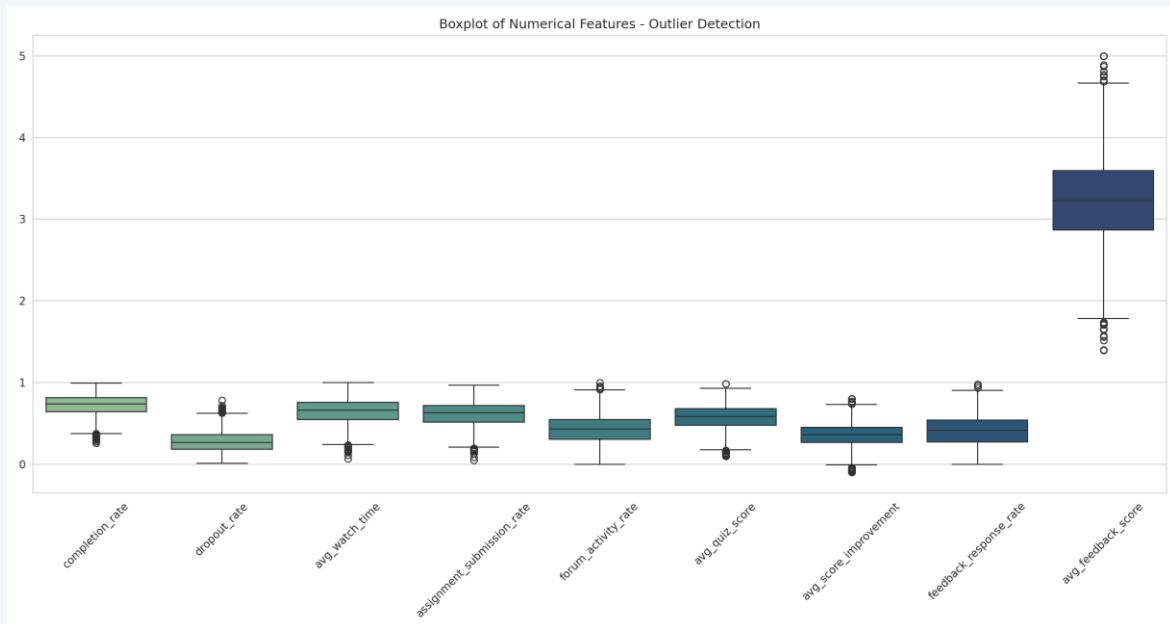


Figure 5.2 - Boxplot of all numerical features, used to flag potential outliers.

A boxplot summarises a distribution using five numbers: the minimum, the first quartile (Q1, 25th percentile), the median (Q2, 50th percentile), the third quartile (Q3, 75th percentile), and the maximum. Points that fall outside the 'whiskers' are flagged as potential outliers using Tukey's rule:

$$\text{IQR} = Q3 - Q1 \quad \text{Lower fence} = Q1 - 1.5 \times \text{IQR} \quad \text{Upper fence} = Q3 + 1.5 \times \text{IQR}$$

Any observation below the lower fence or above the upper fence is drawn as an individual point. In this dataset, the boxplots show only a small number of such points across several features, and these appear to represent natural variation in instructor and student performance rather than data entry errors - so no additional outlier removal was applied.

5.3 Correlation Heatmap

```
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt='.2f')
```

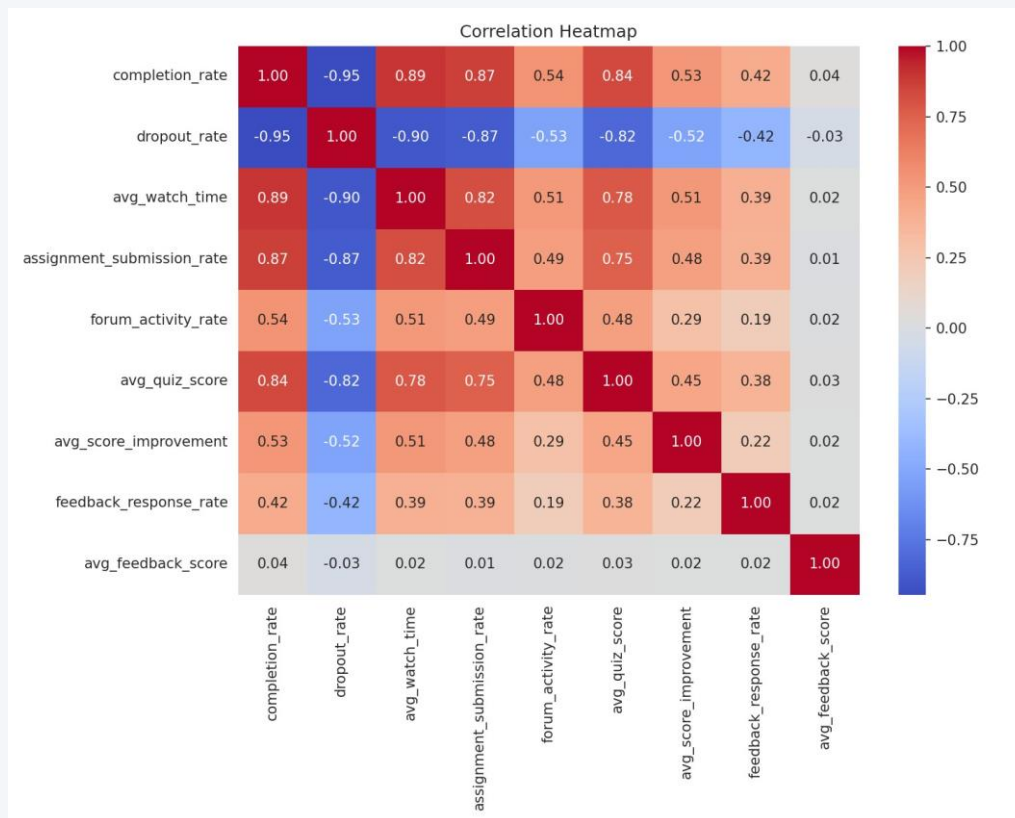


Figure 5.3 - Pearson correlation heatmap across all numeric features.

Each cell in the heatmap is the Pearson correlation coefficient between a pair of variables:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{[\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2]}}$$

r ranges from -1 (perfect negative relationship) to +1 (perfect positive relationship), with 0 meaning no linear relationship. The heatmap confirms that completion_rate and dropout_rate are strongly negatively correlated (they are two sides of the same coin), and that engagement-oriented features (watch time, quiz score, assignment submission) correlate positively with completion_rate. avg_feedback_score, however, shows much weaker correlation with every other variable - an early signal, confirmed later by the modelling results, that instructor feedback is not well explained by behavioural engagement alone.

5.4 Bivariate Pairplot Analysis

```
sns.pairplot(df[['completion_rate', 'dropout_rate', 'avg_quiz_score', 'avg_watch_time', 'avg_feedback_score']])
```

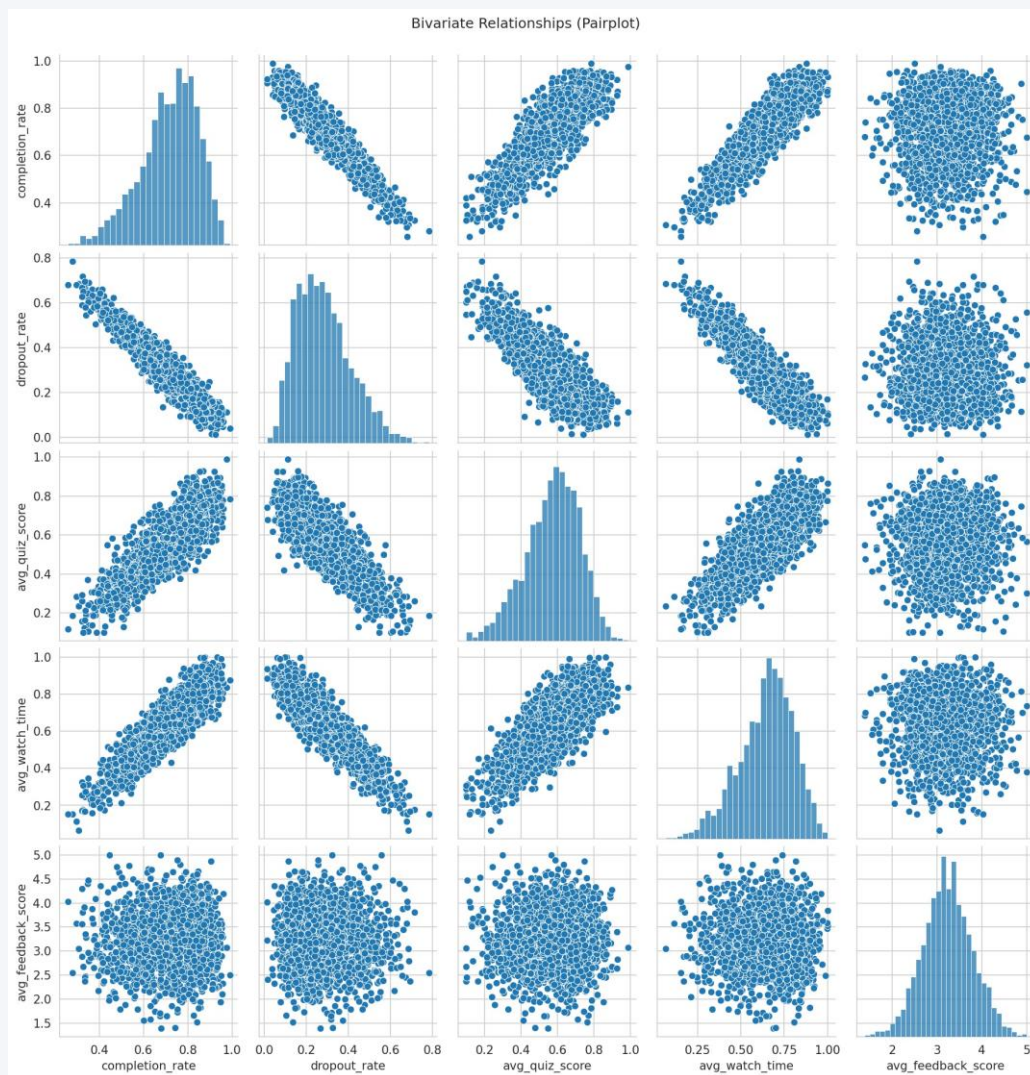


Figure 5.4 - Pairwise scatterplots and diagonal histograms for five key metrics.

A pairplot arranges a scatterplot for every pair of chosen variables in a grid, with each variable's own histogram along the diagonal. This lets many bivariate relationships be screened in a single figure. Clear negative-sloping clouds appear between completion_rate and dropout_rate, while completion_rate vs avg_quiz_score and avg_watch_time show positive, fairly tight clouds - visual confirmation of the correlations already seen in the heatmap. The relationship between completion_rate and avg_feedback_score is comparatively diffuse, again pointing to weaker linear structure.

5.5 Regression Plots - Completion Rate Drivers

Each regplot below fits a simple linear regression line ($y = mx + c$, via OLS) through a scatter of one predictor against completion_rate, with a shaded 95% confidence band around the line. This isolates the marginal, one-to-one relationship between a single feature and the target, before the features are combined in a full multi-variable model.

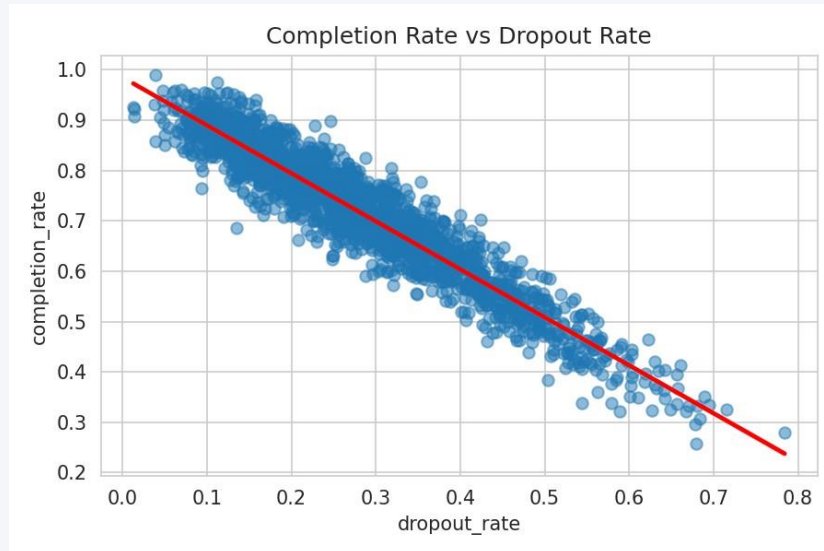


Figure 5.5a - Completion Rate vs Dropout Rate. A steep negative slope confirms these two metrics are structurally linked.

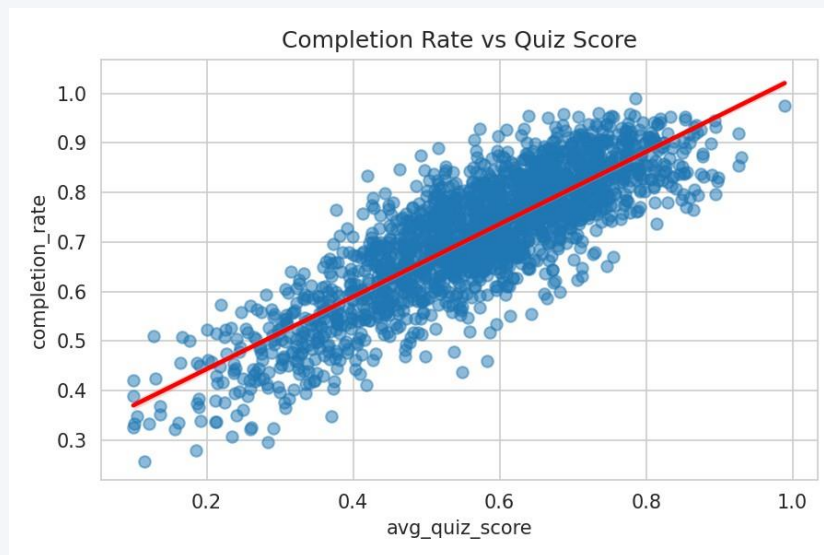


Figure 5.5b - Completion Rate vs Average Quiz Score. A clear positive slope: stronger quiz performers are more likely to complete the course.



Figure 5.5c - Completion Rate vs Score Improvement. A weaker, noisier positive trend, suggesting improvement over time matters less than absolute quiz performance.

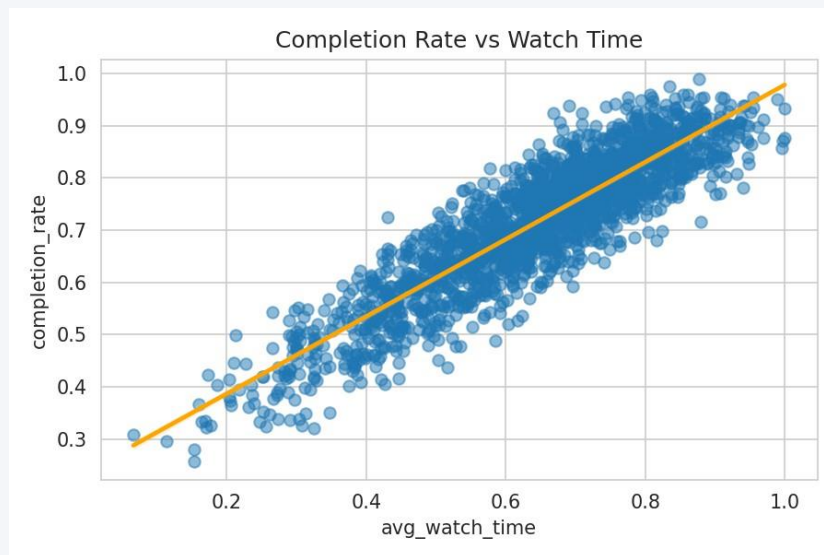


Figure 5.5d - Completion Rate vs Watch Time. One of the strongest single-feature relationships in the dataset - watch time is a strong proxy for overall engagement.

Across all four regplots, the slope coefficient m is estimated the same way as in the full model - by minimising squared residuals - but using only one predictor at a time. Comparing these single-variable slopes to the multi-variable coefficients in Section 8 shows how much of each feature's apparent effect survives once the other features are controlled for.

5.6 Course-wise Completion Rate Comparison

```
sns.boxplot(x='course_id', y='completion_rate', data=df)
```

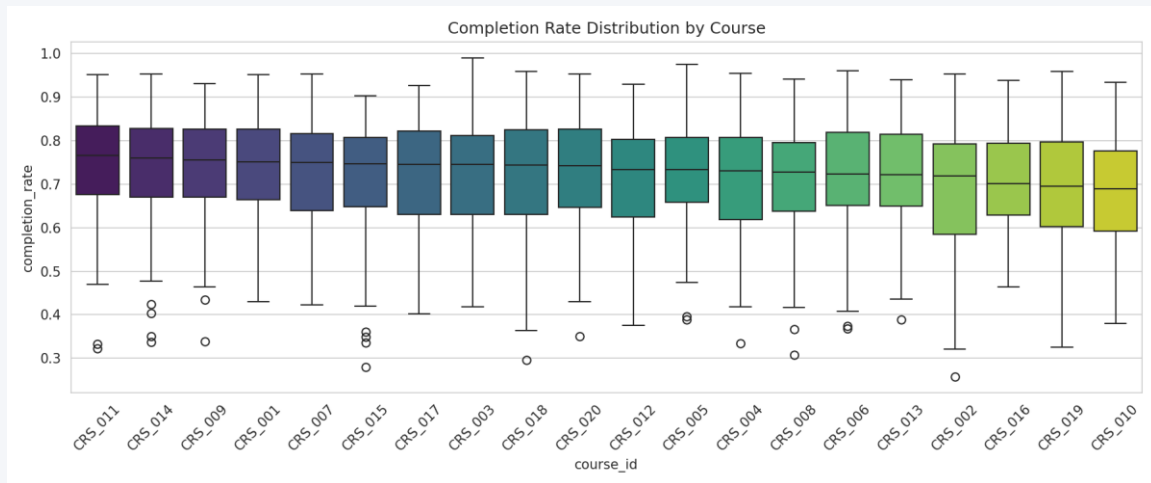


Figure 5.6 - Completion rate distribution for each course, sorted by median.

Grouping `completion_rate` by `course_id` and plotting a boxplot per group reveals whether some courses systematically under- or out-perform others, independent of any single instructor or batch. Courses whose boxes sit noticeably lower than the rest - with medians and interquartile ranges shifted downward - are strong candidates for a curriculum or content-design review, since the effect persists across the students and instructors assigned to that course.

5.7 Pairplot with Regression Fits

```
sns.pairplot(df[[...]], kind='reg', plot_kws={'scatter_kws': {...}, 'line_kws': {...}})
```

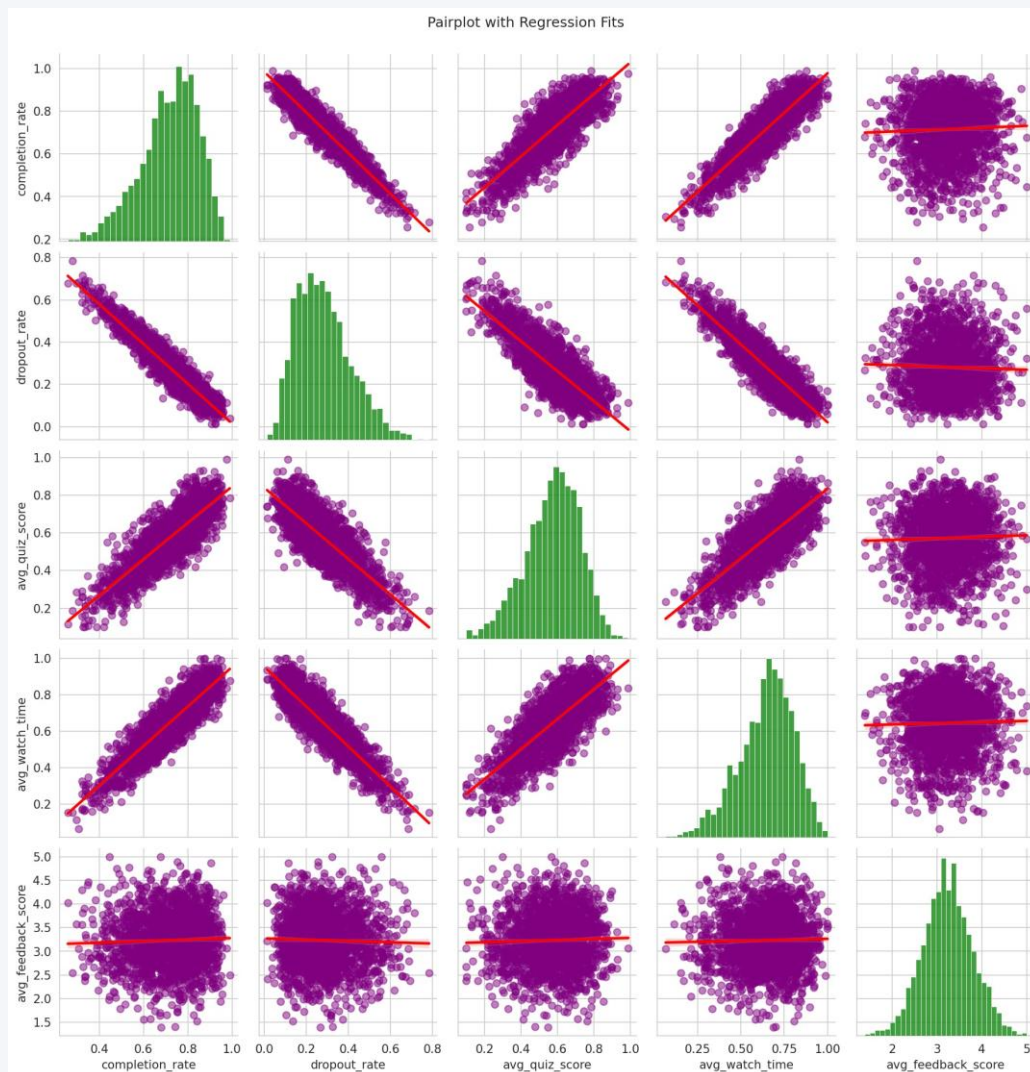


Figure 5.7 - The same five-variable pairplot as Figure 5.4, now with an OLS regression line overlaid on every off-diagonal panel.

Adding `kind='reg'` overlays a best-fit regression line on every scatter panel, turning a purely visual exploration into a quantitative one: the steepness and direction of each red line is a direct visual proxy for that pair's correlation coefficient. This figure acts as a bridge between the exploratory phase and the modelling phase - the same pairwise relationships visualised here are what the multi-variable regression models in Sections 8-10 will attempt to combine and disentangle simultaneously.

6. Feature Engineering

Feature engineering is the process of constructing new variables from existing ones so that a model can learn patterns more easily. Two composite features were engineered in this project, each averaging together raw metrics that share a common underlying theme.

6.1 engagement_score

```
engagement_score = (avg_watch_time + assignment_submission_rate +  
forum_activity_rate) / 3
```

This feature averages three behavioural signals - how much video a student watches, how reliably they submit assignments, and how active they are in forum discussion - into a single 0-1 score representing overall behavioural engagement. Averaging (rather than, say, summing) keeps the resulting feature on the same 0-1 scale as its inputs, which keeps it directly comparable and easy to interpret.

6.2 learning_performance

```
learning_performance = (avg_quiz_score + avg_score_improvement) / 2
```

This feature combines a student's absolute quiz performance with how much that performance improved over time, producing a single measure of learning outcome that rewards both high achievement and positive trajectory.

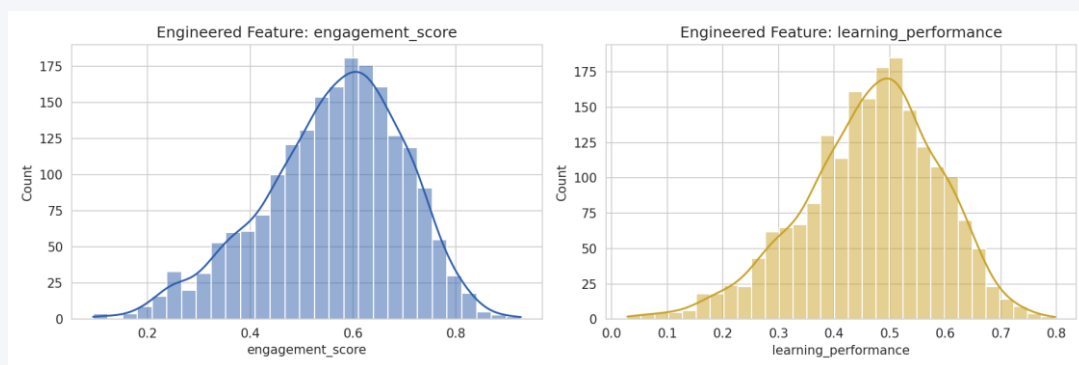


Figure 6.1 - Distributions of the two engineered features, *engagement_score* and *learning_performance*.

6.3 Why Engineer Features at All?

Three reasons motivate this step. First, **dimensionality reduction with meaning preserved**: three or two raw columns become one, reducing redundancy while keeping an interpretable label. Second, **noise averaging**: averaging correlated signals cancels out some of each individual signal's measurement noise, since independent noise terms partially offset one another when summed. Third, **business interpretability**: 'engagement_score' and 'learning_performance' are concepts a course administrator can reason about directly, which makes the eventual model outputs easier to explain to non-technical stakeholders.

Both engineered features were then added alongside the original raw columns as inputs to the regression models described in the following sections - giving each model the choice of using the raw signals, the composite signals, or both.

7. Machine Learning Methodology

With clean, feature-engineered data in hand, the project moves to building predictive models. The same disciplined process is repeated for each of the three target variables.

7.1 Train / Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

The dataset is randomly split into 80% for training and 20% for testing. The model learns its coefficients only from the training rows; the test rows are held back and used purely to evaluate how well the model generalises to data it has never seen. Without this split, a model could simply memorise the training data and appear deceptively accurate while failing on new records. Setting `random_state=42` fixes the random shuffling seed so the split - and therefore the reported metrics - are exactly reproducible.

7.2 Model Choice: Linear Regression

scikit-learn's `LinearRegression` fits an Ordinary Least Squares model: it solves directly for the coefficient vector β that minimises the sum of squared residuals across the training set. In matrix form:

$$\beta = (X^T X)^{-1} X^T y$$

where X is the matrix of training features (with a column of 1s prepended for the intercept) and y is the vector of target values. No iterative optimisation or hyperparameter tuning is required - the solution is computed in one closed-form step, which is part of why Linear Regression is such a fast, reliable baseline.

7.3 Feature Sets per Target

For each target variable, every other numeric column (including the two engineered features) was used as a predictor, with the target itself excluded from its own feature list. This 'everything else predicts this one thing' setup is a natural way to probe how much of a metric's variation can be explained by all the other available signals combined.

Target	Predictors Used
completion_rate	dropout_rate, avg_score_improvement, avg_quiz_score, avg_watch_time, assignment_submission_rate, forum_activity_rate, avg_feedback_score, feedback_response_rate, engagement_score, learning_performance
avg_feedback_score	completion_rate, dropout_rate, avg_score_improvement, avg_quiz_score, avg_watch_time, assignment_submission_rate, forum_activity_rate, feedback_response_rate, engagement_score, learning_performance
dropout_rate	completion_rate, avg_score_improvement, avg_quiz_score, avg_watch_time, assignment_submission_rate, forum_activity_rate, avg_feedback_score, feedback_response_rate, engagement_score, learning_performance

8. Model 1: Predicting Completion Rate

The first model predicts `completion_rate` from the remaining behavioural and engineered features. It achieves an R^2 of 0.921, meaning roughly 92.1% of the variation in completion rate across students is explained by the model - a strong result for a linear model, and consistent with the tight, near-linear patterns already visible in the Section 5 scatterplots.

Metric	Value	Meaning
MAE	0.0292	Average absolute prediction error, in the target's own units
RMSE	0.0365	Root-mean-square error; penalises large errors more than MAE
R^2 Score	0.9210	Proportion of variance in the target explained by the model

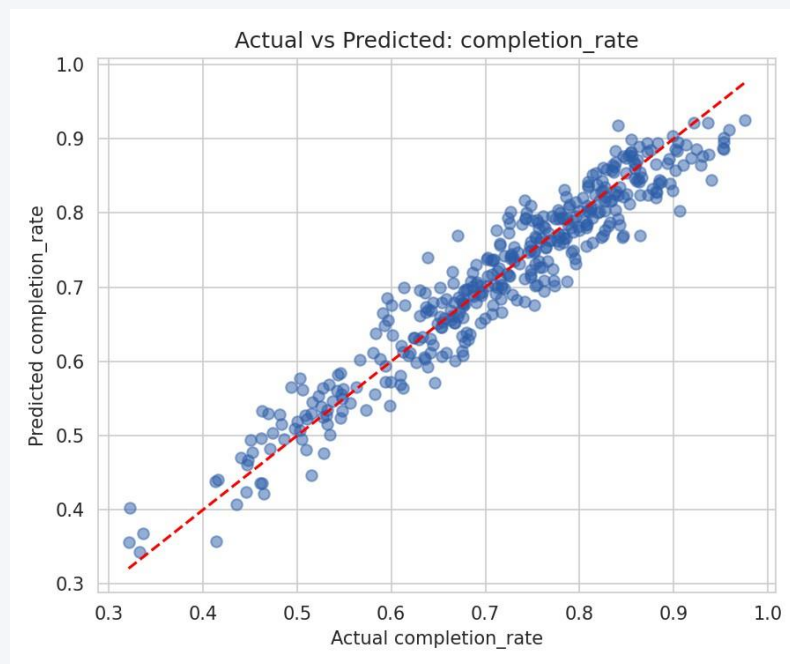


Figure - Actual vs Predicted `completion_rate`. Points close to the diagonal red dashed line indicate accurate predictions.

Coefficient Interpretation - completion_rate

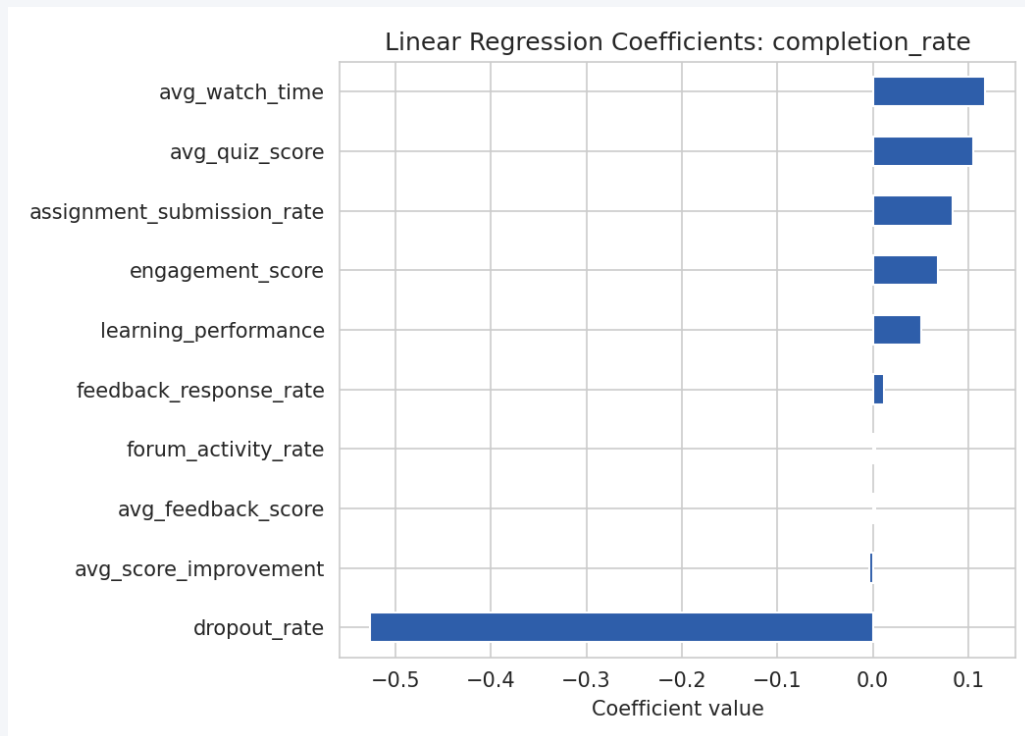


Figure - Linear regression coefficients, sorted by value. Bars extending right increase the prediction; bars extending left decrease it.

As expected from the correlation heatmap, `dropout_rate` carries by far the largest (negative) coefficient, since a higher dropout rate mechanically implies a lower completion rate. Among the remaining behavioural predictors, `avg_watch_time`, `avg_quiz_score` and `assignment_submission_rate` contribute the next largest positive weights - reinforcing that consistent viewing, strong quiz performance, and reliable assignment submission are the clearest behavioural levers for improving completion.

9. Model 2: Predicting Avg. Feedback Score

The second model attempts to predict avg_feedback_score from behavioural and engagement metrics. Its R^2 of -0.018 is far lower than the completion-rate model - the behavioural features explain very little of the variation in how students rate their instructor. This is itself a meaningful finding, not a modelling failure: instructor feedback is a subjective, largely independent signal, likely shaped by factors such as communication style, responsiveness, and rapport that are not captured anywhere in this behavioural dataset.

Metric	Value	Meaning
MAE	0.4391	Average absolute prediction error, in the target's own units
RMSE	0.5503	Root-mean-square error; penalises large errors more than MAE
R^2 Score	-0.0182	Proportion of variance in the target explained by the model

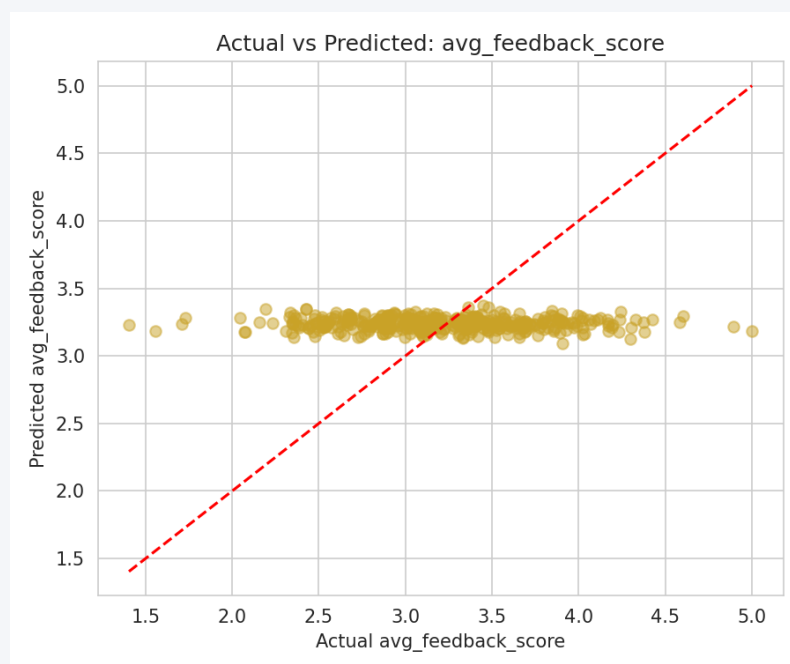


Figure - Actual vs Predicted avg_feedback_score. Points close to the diagonal red dashed line indicate accurate predictions.

Coefficient Interpretation - avg_feedback_score

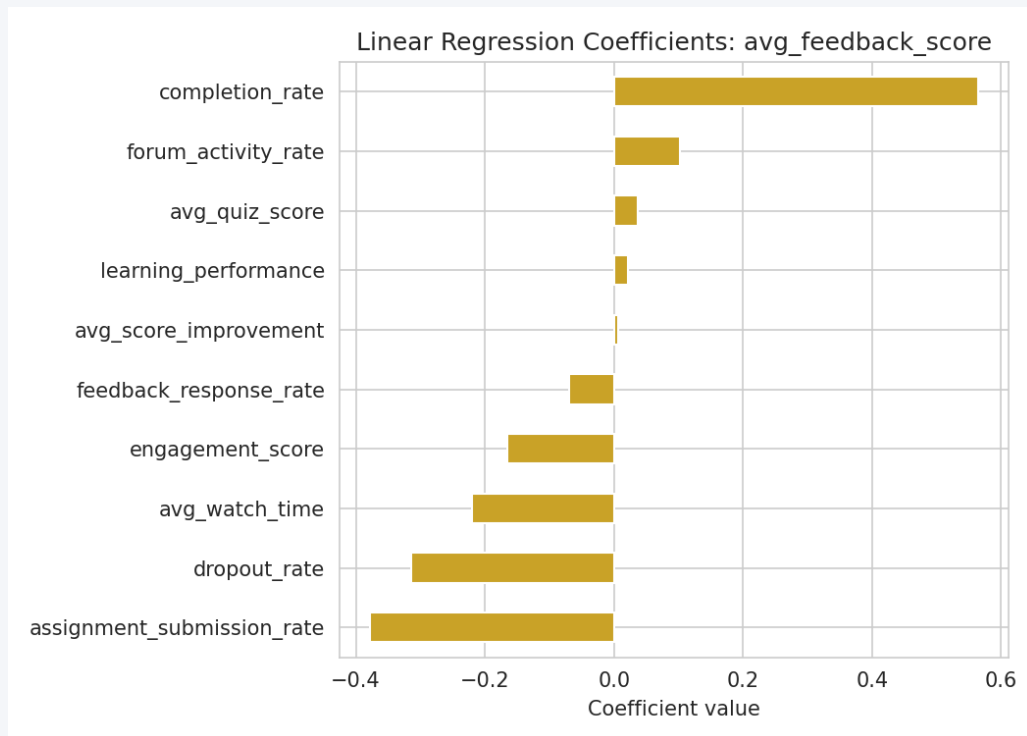


Figure - Linear regression coefficients, sorted by value. Bars extending right increase the prediction; bars extending left decrease it.

The coefficient plot shows no single dominant driver: weights are comparatively small and spread across many features, with completion_rate itself contributing the largest positive coefficient. The practical takeaway is that improving feedback scores likely requires qualitative interventions - instructor training, communication coaching, or direct sentiment analysis of written feedback - rather than optimisation of behavioural engagement metrics.

10. Model 3: Predicting Dropout Rate

The third model mirrors Model 1, this time predicting dropout_rate directly. It achieves an R^2 of 0.923, on par with the completion-rate model - unsurprising given how strongly the two targets are linked, but useful in its own right for building an early-warning system that flags at-risk batches before they finish (or fail to finish) a course.

Metric	Value	Meaning
MAE	0.0285	Average absolute prediction error, in the target's own units
RMSE	0.0351	Root-mean-square error; penalises large errors more than MAE
R^2 Score	0.9235	Proportion of variance in the target explained by the model

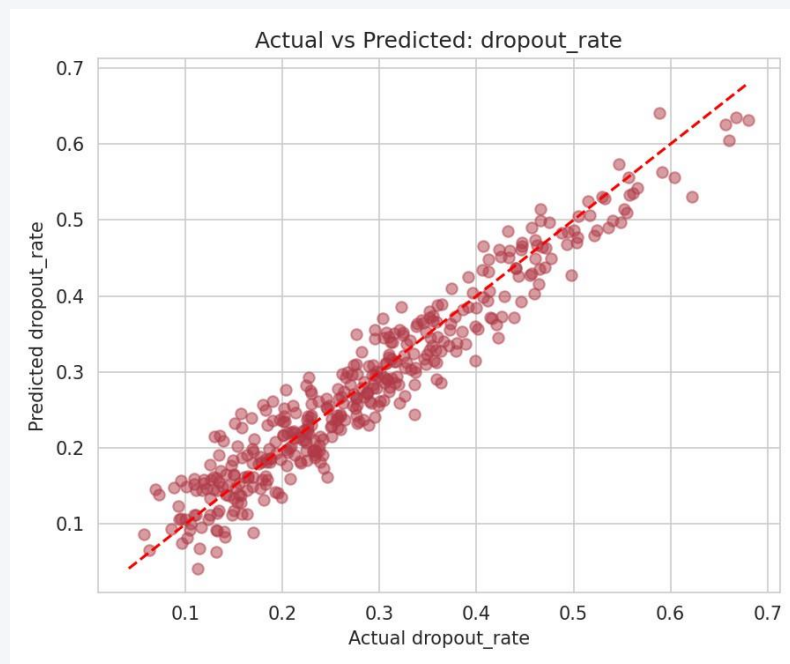


Figure - Actual vs Predicted dropout_rate. Points close to the diagonal red dashed line indicate accurate predictions.

Coefficient Interpretation - dropout_rate

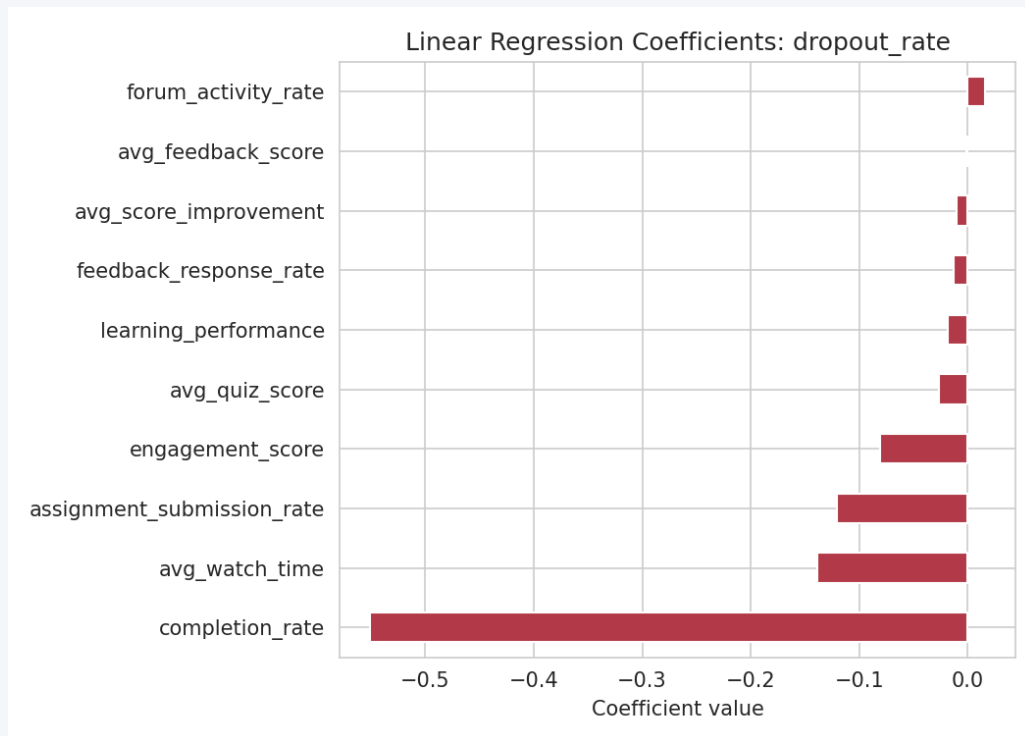


Figure - Linear regression coefficients, sorted by value. Bars extending right increase the prediction; bars extending left decrease it.

completion_rate again dominates the coefficient plot (with the expected negative sign), followed by avg_watch_time and assignment_submission_rate, both negative - i.e. more watch time and more submitted assignments are associated with lower dropout. This mirrors the completion-rate findings from the opposite direction and strengthens confidence that watch time and assignment submission are the two most actionable levers available to course administrators.

11. Model Evaluation Metrics Explained

All three models were judged using the same three regression metrics. Each answers a slightly different question about model quality.

11.1 Mean Absolute Error (MAE)

$$\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$$

MAE is the average size of the prediction error, in the same units as the target variable, with no special penalty for large errors. An MAE of 0.03 on `completion_rate`, for example, means predictions are typically off by about 3 percentage points.

11.2 Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{[(1/n) \sum (y_i - \hat{y}_i)^2]}$$

RMSE squares each error before averaging, so larger errors are penalised disproportionately more than small ones, then takes the square root to bring the units back to the original scale. RMSE is always $\geq$ MAE; the closer the two are, the more uniform the model's errors are across all predictions.

11.3 R² Score (Coefficient of Determination)

$$R^2 = 1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2]$$

R² compares the model's squared errors to the squared errors of a naive model that always predicts the mean, $\bar{y}$. An R² of 1.0 means perfect prediction; an R² of 0 means the model is no better than always guessing the average; a negative R² (as seen in the feedback-score model) means the model performs worse than that naive average baseline - a strong signal that the chosen features simply don't explain that target well.

Used together, these three metrics give a rounded picture: MAE for typical error size, RMSE for sensitivity to large mistakes, and R² for overall explanatory power relative to a naive baseline.

12. Business Insights & Recommendations

Completion and dropout are two sides of the same coin.

Because these targets are highly predictable ($R^2 > 0.9$) from the same handful of engagement metrics, a single early-warning dashboard tracking watch time, assignment submission, and quiz performance can flag at-risk batches for both outcomes simultaneously.

Watch time and assignment submission are the strongest actionable levers.

Both appear as top-magnitude coefficients in the completion and dropout models. Course teams should prioritise interventions that increase video engagement and submission compliance - e.g. shorter video modules, submission reminders, or micro-deadlines.

Instructor feedback score needs its own, separate strategy.

Behavioural engagement data barely explains feedback scores. Improving instructor ratings likely requires qualitative approaches: teaching-style training, communication coaching, or analysing the free-text content of student feedback rather than clickstream metrics.

Course-level effects exist independent of instructor or batch.

The course-wise boxplot (Figure 5.6) shows some courses with systematically lower completion medians - a signal to review course content and structure, not just instructor performance, for those specific courses.

Engineered features add interpretability without losing information.

`engagement_score` and `learning_performance` summarise related raw metrics into business-friendly composites, making it easier to communicate model findings to non-technical stakeholders such as programme managers.

13. Conclusion & Future Scope

This project demonstrates a complete, well-structured pipeline from raw instructor effectiveness data to actionable insight: rigorous preprocessing, thorough exploratory analysis across ten-plus visualisations, thoughtful feature engineering, and three interpretable regression models evaluated on held-out data. The results show that student completion and dropout are both strongly explainable from behavioural engagement metrics, while instructor feedback is a largely independent signal requiring separate qualitative attention.

13.1 Suggested Next Steps

- Test non-linear models (Random Forest, Gradient Boosting) to check for interaction effects the linear models cannot capture, especially for the harder-to-predict feedback score.
- Apply regularisation (Ridge / Lasso regression) if multicollinearity between engineered and raw features is a concern, since `engagement_score` and `learning_performance` are by construction correlated with their own input columns.
- Incorporate text-based sentiment analysis of written student feedback to add a genuinely new signal for the feedback-score model.
- Build a live dashboard that scores each active batch weekly against the completion / dropout models, surfacing at-risk batches to programme managers in real time.
- Validate the models on a rolling basis as new cohorts complete courses, to check whether the relationships identified here hold stable over time.

Overall, the project shows how a disciplined, well-documented data science workflow can convert raw learning-platform exhaust into decision-grade intelligence that supports better teaching strategy, improved course quality, and stronger student outcomes.

A. Appendix - Full Code Reference

For quick reference, the complete code for each stage of the pipeline described in this handbook is reproduced below, grouped by stage.

A.1 Loading & Preprocessing

```
import numpy as np
import pandas as pd

df = pd.read_csv('/content/dataset.csv')
display(df.head())

df = df.drop_duplicates()
df['avg_score_improvement'] = df['avg_score_improvement'].fillna(
    df['avg_score_improvement'].median())
```

A.2 Exploratory Data Analysis

```
import matplotlib.pyplot as plt
import seaborn as sns

df.hist(figsize=(15,10)); plt.tight_layout(); plt.show()

plt.figure(figsize=(15,8))
sns.boxplot(data=df.select_dtypes(include='number'))
plt.xticks(rotation=45); plt.show()

plt.figure(figsize=(10,8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap'); plt.show()

sns.pairplot(df[['completion_rate', 'dropout_rate', 'avg_quiz_score',
                'avg_watch_time', 'avg_feedback_score']])

sns.regplot(x='dropout_rate', y='completion_rate', data=df,
            scatter_kws={'alpha':0.5}, line_kws={'color':'red'})

sns.boxplot(x='course_id', y='completion_rate', data=df)
```

A.3 Feature Engineering

```
df['engagement_score'] = (  
    df['avg_watch_time'] + df['assignment_submission_rate']  
    + df['forum_activity_rate']  
) / 3  
  
df['learning_performance'] = (  
    df['avg_quiz_score'] + df['avg_score_improvement']  
) / 2
```

A.4 Model Training & Evaluation (repeated per target)

```
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score  
  
X = df[features]  
y = df[target]  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42)  
  
lr = LinearRegression()  
lr.fit(X_train, y_train)  
y_pred = lr.predict(X_test)  
  
mae = mean_absolute_error(y_test, y_pred)  
rmse = np.sqrt(mean_squared_error(y_test, y_pred))  
r2 = r2_score(y_test, y_pred)  
  
print('MAE:', mae)  
print('RMSE:', rmse)  
print('R2 Score:', r2)
```

A.5 Glossary of Key Terms

Term	Definition
Median imputation	Filling missing values with the middle value of a sorted column
Duplicate row	An exact repeat of another record, typically removed before analysis
Pearson correlation (r)	A -1 to +1 measure of linear association between two variables
OLS (Ordinary Least Squares)	The method Linear Regression uses to fit coefficients that minimise squared error
Train/test split	Dividing data so model fit and model evaluation use different rows
MAE / RMSE / R ²	The three regression metrics used to judge model accuracy in this handbook